Universidad Veracruzana

Facultad de Instrumentación Electrónica

Ingeniería Biomédica

EE: Interfaces Hombre-Máquina

ACTIVIDAD 05. SENSOR PAJ7620 COMO DISPOSITIVO PARA UNA INTERFAZ HOMBRE-MAQUINA SIN CONTACTO

Profesor:
Varela Lara Luis Julian

Integrantes del equipo *"ES3.14"*:
Alfonso Gamboa Rubén
Flores Montero Edsel Yetlanezi

Fecha: 20 de mayo de 2026

Xalapa, Enríquez, Ver.

I. Introducción al tema.

El objetivo principal es establecer un sistema físico que permita interactuar en tiempo real con el programa “ComandosRS232”, traduciendo movimientos direccionales de la mano en comandos de navegación para una interfaz gráfica.

A diferencia de los proyectos previos, este diseño aprovecha el reconocimiento óptico de gestos para ejecutar acciones direccionales (arriba, abajo, izquierda, derecha) de forma continua, así como una acción de selección o detención mediante el gesto de “acercar” simulando el click de un cursor. Otros temas mencionados anteriormente, serán omitidos para evitar redundancia.

I-A. Marco teórico

I-A1. Sensor de Reconocimiento de Gestos PAJ7620: El PAJ7620U2 [1], [2] es un sensor óptico integrado diseñado específicamente para el reconocimiento de gestos en interfaces hombre-máquina (HMI). A diferencia de los sensores qué miden el cambio de posición midiendo gravedad y aceleración, este dispositivo está compuesto internamente por un diodo emisor de luz infrarroja (IR LED) y un banco de sensores ópticos especializados que detectan el reflejo de la luz cuando un objeto pasa frente a él.

Esté dispositivo opera con voltajes compatibles con el ESP32 y se comunica utilizando el protocolo I2C (Inter-Integrated Circuit). Su mayor ventaja radica en su capacidad de reconocer hasta 9 gestos predefinidos (Arriba, Abajo, Izquierda, Derecha, Acercar, Alejar, etc.). Esto libera al microcontrolador maestro de realizar cálculos complejos o aplicar filtros de señal para limpiar el ruido, y simplifica las instrucciones necesarias para su operación.

1. Emisión (La “Linterna” Infrarroja): El módulo integra un Diodo Emisor de Luz (LED) infrarroja. Este LED está parpadeando miles de veces por segundo, emitiendo un cono de luz hacia arriba que el ojo humano no puede detectar, pero el sensor sí.
2. Reflexión (El Rebote): Cuando se pasa un objeto por encima del sensor (generalmente a una distancia de 5 a 15 cm), actúa como un espejo imperfecto. Esto interrumpe ese cono de luz y refleja una porción de los rayos infrarrojos de regreso hacia abajo, hacia la lente del sensor.
3. Recepción (La Matriz de Píxeles): Debajo de la lente del PAJ7620 hay un arreglo de fotodiodos especializada en luz infrarroja. Cuando el reflejo golpea esta matriz, ilumina un grupo específico de píxeles, activando una zona brillante en el sensor.
4. Procesamiento (El Algoritmo Interno): El chip tiene un procesador interno que toma “fotografías” de esta zona brillante a altísima velocidad y analiza su cambio:
    - Dirección (Arriba/Abajo/Izquierda/Derecha): Si en el fotograma 1 la mancha brillante está en el lado izquierdo de la matriz, y en el fotograma 2 se ha movido al lado derecho, el sensor entiende que físicamente se movió de izquierda a derecha (GES_RIGHT).
    - Profundidad (Acercar/Alejar): Aquí radica la física del comando GES_FORWARD (Acercar). Si la mancha de luz se vuelve más grande y brillante, significa que el objeto que refleja la luz está más cerca de la lente. Si la mancha se encoge y atenúa, se está alejando (GES_BACKWARD).

I-A2. Librería RevEng_PAJ7620: Para la integración del hardware y la extracción de datos se utilizó la biblioteca de código abierto RevEng_PAJ7620 ; versión compatible para ESP32 y Arduino UNO. Esta librería emplea comunicación I2C (es decir con los pines SDA y SCL) y facilita la interpretación de los registros del sensor [3], [4] . A diferencia de los sensores donde el microcontrolador debe procesar los datos, el PAJ7620 delega esta tarea al propio hardware del dispositivo. La librería RevEng_PAJ7620 actúa como un puente que simplifica la obtención de datos y la manipulación de esta información procesada por el mismo sensor.

Cuando el arreglo óptico del PAJ7620 detecta un gesto, mediante su algoritmo interno guarda un código hexadecimal específico para cada dirección en la qué se detecta la refracción óptica del infrarrojo en su banco de registros, y los almacena directamente en la función sensor.readGesture(). Al llamar a la función de lectura, la librería extrae el valor del registro y lo devuelve convertido en un tipo de dato estructurado (Gesture). Esto significa que el código principal del ESP32 trabaja con etiquetas legibles por humanos (como GES_UP, GES_DOWN, GES_FORWARD, etc) en lugar de datos crudos.

El diseño del sistema aprovecha el pin de interrupción del sensor. El microcontrolador no necesita verificar el estado del sensor (haciendo polling I2C) para saber si hubo un movimiento. El código simplemente monitorea un pin digital (PIN_INT 15); cuando el sensor registra un gesto válido, define este pin a estado LOW (0). Solo en ese instante se ejecuta la lectura de la librería, ahorrando ciclos de reloj en el ESP32.

Gracias a que la librería entrega gestos limpios y definidos, esto permite la implementación de una lógica basada en la función switch-case. por lo qué se le puede considerar una “Máquina de Estados Finitos”, simplificando las instrucciones de movimiento reflejadas en la interfaz del usuario.

La simplificación matemática que provee la librería permite dividir el ciclo principal (loop) en dos hilos lógicos asíncronos:

Lectura y actualización de estado: Qué se encarga exclusivamente de detectar el gesto con readGesture() y actualizar hacia dónde debe ir el cursor.

Transmisión continua: Un bloque independiente que evalúa la variable estadoActual y envía las tramas de control correspondientes por puerto Serial cada 24 ms utilizando millis().

II. Desarrollo de la actividad.

II-A. Librerías.

Primeramente se revisó el código de ejemplo “Prueba-PAJ7620.ino”, el cual es un código de ejemplo de uso del Sensor PAJ7620. Dentro de este código de ejemplo podemos encontrar la librería “RevEng_PAJ7620.h” que como vimos

en la sección I-A2, ésta permite perfectamente compatibilidad con nuestro microcontrolador *ESP32*. Con la diferencia que se trabajan distintos pines a los de un arduino uno.

II-B. Desarrollo del código.

Como buena práctica, se declaran variables que nos pueden servir de ajustes maestros al inicio del código. El tiempo se declara con *unisgned long* debido a que utilizaremos la función *milis()* para poder iterar dentro de nuestro código sin usar *delay()*; por lo que al no trabajar con *un tiempo negativo*, podemos almacenar números maś grandes. Para un entendimiento más fácil del código, se creó una maquina de estados [5] para poder controlar de forma más sencilla la dirección a la que queremos que se mueva nuestro cursor.

II-B1. setup(): Dentro del setup iniciamos la comunicación serial con *115200* baudios, estándar con el ESP32, y activamos el LED cuando se inicia la conexión con el sensor.

II-B2. loop(): Para el desarrollo de esta actividad se mantuvo una lógica similar a las actividades anteriores, donde se evitó el uso de *delay()*, registrando el tiempo actual y se evalúa si ha transcurrido el tiempo necesario para ejecutar el algoritmo. Gran parte del trabajo se facilitó por la eficiencia de la librería anteriormente mencionada I-A2, ya que permite extraer los datos que queremos solo con la función *sensor.readGesture()* y con la *máquina de estados* consultar la dirección dirección del gesto, y con un *switch* cambiar la misma. EL LED se enciende y se apaga al detectar las lecturas del sensor.

Si el algorimo detecta que el gesto de acercarse se realiza dos veces consecutivas, este actúa como una selección (imprimir 50); si se detecta una sola vez este se interpreta como una señal de stop. Se eligió este comportamiento debido a su similitud con el comportamiento que tiene un ratón de computadora, haciéndolo intuitivo y conocido para el usuario.

Finalmente, dentro de cada ciclo, el código lee la máquina de estados, y según el mismo, define la dirección del cursor (se imprime 10, 20, 30 o 40 según sea el caso).

II-C. Código completo

A continuación se incluye el código completo empleado para este sistema de interfaz Hombre-Máquina:

```
#include "RevEng_PAJ7620.h"

#define PIN_INT   15
#define PIN_LED   26

// —— Variables modificables fácilmente
// Frecuencia de envío (en milisegundos).
unsigned long intervalo_envio = 24;
// Tiempo máximo (en ms) entre los dos gestos de
↪  "acercar" para mandar el recorte
const unsigned long TIEMPO_MAX_DOBLE_ACERCAR =
↪  2000;

RevEng_PAJ7620 sensor = RevEng_PAJ7620();
Gesture gesto;

unsigned long tiempoAnterior = 0;
const unsigned long DEBOUNCE = 400;
unsigned long tiempoUltimoEnvio = 0;

// Máquina de estados para las direcciones
enum Estado { STOP, ARRIBA, ABAJO, IZQUIERDA,
↪  DERECHA };
Estado estadoActual = STOP;

// Variables para controlar el doble gesto
bool esperandoSegundoAcercar = false;
unsigned long tiempoUltimoAcercar = 0;

void setup()
{
  Serial.begin(115200);
  pinMode(PIN_INT, INPUT_PULLUP);
  pinMode(PIN_LED, OUTPUT);

  if( !sensor.begin() )
  {
    digitalWrite(PIN_LED, HIGH); // prendemos el LED
    ↪  si hay conexión
  }
}

void loop()
{
  unsigned long ahora = millis();

  // 1. LECTURA DE GESTOS
  if(digitalRead(PIN_INT) == 0)
  {
    if(ahora - tiempoAnterior >= DEBOUNCE)
    {
      tiempoAnterior = ahora;
      digitalWrite(PIN_LED, HIGH);

      gesto = sensor.readGesture();

      switch(gesto)
      {
        case GES_UP:
          esperandoSegundoAcercar = false; //
          ↪  Cancela la espera del doble gesto
          if(estadoActual == ABAJO) estadoActual =
          ↪  STOP;
          else estadoActual = ARRIBA;
          break;

        case GES_DOWN:
          esperandoSegundoAcercar = false;
          if(estadoActual == ARRIBA) estadoActual =
          ↪  STOP;
          else estadoActual = ABAJO;
          break;

        case GES_LEFT:
          esperandoSegundoAcercar = false;
          if(estadoActual == DERECHA) estadoActual
          ↪  = STOP;
          else estadoActual = IZQUIERDA;
          break;

        case GES_RIGHT:
          esperandoSegundoAcercar = false;
          if(estadoActual == IZQUIERDA)
          ↪  estadoActual = STOP;
```

```
          else estadoActual = DERECHA;
          break;

        // --- LÓGICA DE DOBLE "ACERCAR" ---
        case GES_FORWARD:
          estadoActual = STOP; // El primer efecto
          ↪  siempre es detener el movimiento

          // Si ya habíamos acercado recientemente y
          ↪  no hemos superado el tiempo límite
          if (esperandoSegundoAcercar && (ahora -
          ↪  tiempoUltimoAcercar ⩽
          ↪  TIEMPO_MAX_DOBLE_ACERCAR)) {
            Serial.write(50);                // Es
            ↪  el segundo acercar consecutivo,
            ↪  envía el comando
            esperandoSegundoAcercar = false;  //
            ↪  Reinicia para necesitar otros 2
            ↪  gestos para el próximo recorte
          } else {
            // Es el primer acercar (o pasó
            ↪  demasiado tiempo desde el último)
            esperandoSegundoAcercar = true;
            tiempoUltimoAcercar = ahora;
          }
          break;

        default: break;
      }

      digitalWrite(PIN_LED, LOW);
    }
  }

  // 2. ENVÍO CONTINUO DE COMANDOS SIN RETARDOS
  if (estadoActual ≠ STOP)
  {
    if (ahora - tiempoUltimoEnvio ⩾
    ↪  intervalo_envio)
    {
      tiempoUltimoEnvio = ahora;

      switch(estadoActual) {
        case ARRIBA:    Serial.write(30); break;
        case ABAJO:     Serial.write(40); break;
        case IZQUIERDA: Serial.write(20); break;
        case DERECHA:   Serial.write(10); break;
        default: break;
      }
    }
  }
}
```

II-D. Armado de Circuito



Figura 1. Vista general del circuito armado

II-E. Conexiones y pines

Las conexiones se realizaron con alambres de cobre calibre 18, ajustados a la distancia necesaria entre cada pin y acomodados por debajo de la ESP32, con la finalidad de reducir los cables sueltos que pudieran interferir con los gestos del usuario. Como puede apreciarse en la figura 2.

- D15 del ESP32 → INT del PAJ7620U2
- D26 del ESP32→ resistencia 220Ω → Anódo (+) del LED
- D21 del ESP32→ SDA del PAJ7620
- 3.3V del ESP32 → VCC del PAJ7620
- GND del ESP32 → GND el PAJ7620 → Cátodo (-) del LED.

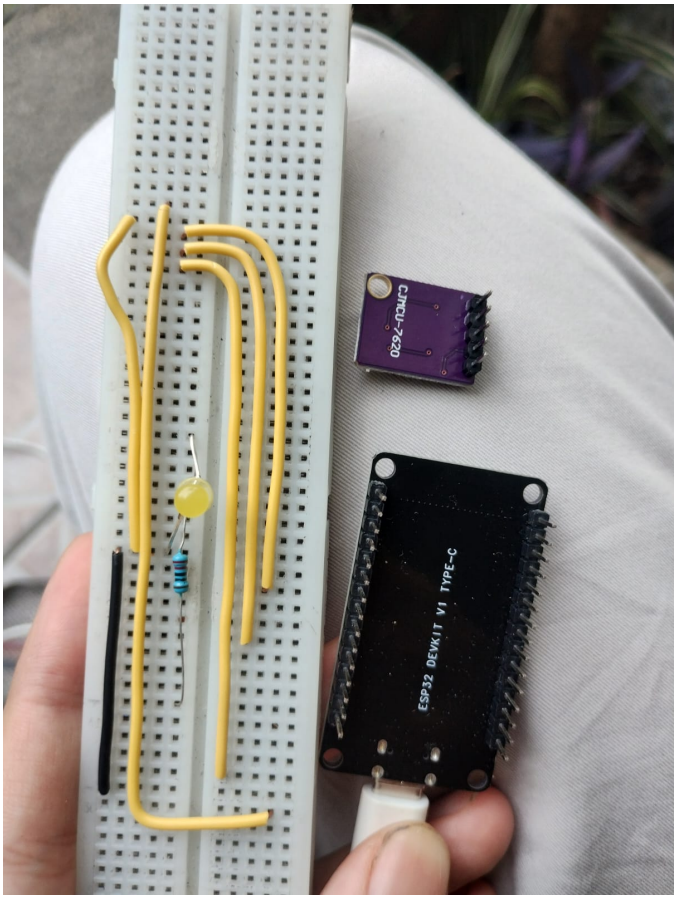


Figura 2. Vista el circuito, desde abajo

III. Conclusiones

A diferencia de actividades previas, el uso de la librería para el sensor simplificó significativamente el desarrollo de esta Interfaz Hombre-Máquina (HMI). Sus funciones sencillas, directas y de alto nivel facilitaron en gran medida la integración lógica dentro del algoritmo final, delegando el procesamiento complejo al hardware.

En cuanto al diseño en físico, la eficiencia del protocolo I2C nos permitió optimizar el uso de los cables. Esto nos llevó a organizar las conexiones de la manera más discreta y compacta posible. Además, optar por alambres cortados a la medida en lugar de cables y montar el sistema sobre una protoboard le otorgó la rigidez estructural necesaria para facilitar su manipulación y evitar interferencias o desconexiones físicas durante la interacción del usuario.

Sobre el diseño del algoritmo, el mayor desafío consistió en ajustar con precisión los comandos de detención y selección. Inicialmente, el sobredesplazamiento natural de la mano al intentar detener el cursor generaba señales erroneas, invirtiendo la dirección en vez de frenarla. Para solucionar esto, se implementó una lógica de control inspirada en el comportamiento de un ratón de computadora (clic y doble clic mediante el gesto de “acercar”). Esta solución no solo eliminó los movimientos indeseados, sino que otorgó mayor fluidez a los desplazamientos durante el cambio de dirección y permitió que las instrucciones del usuario se ejecutaran de forma mucho más definida y controlada.

IV. firma de revisión

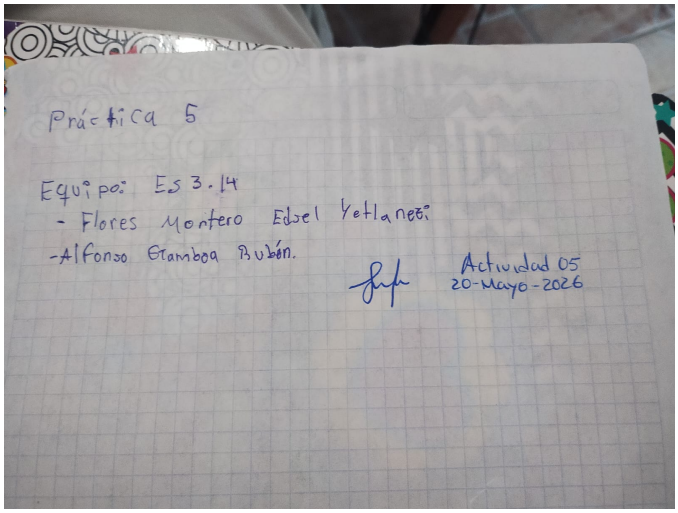


Figura 3. Firma de revisión

Referencias

[1] P. I. Inc., *PAJ7620U2 Integrated Gesture Recognition Sensor Datasheet*, Consultado el 20 de mayo de 2026, 2022. dirección: https://files.seeedstudio.com/wiki/Grove_Gesture_V_1.0/res/PAJ7620U2_DS_v1.5_05012022_Confidential.pdf

[2] MakerGuides. «Sensor de gestos PAJ7620U2 con Arduino.» Consultado el 20 de mayo de 2026. dirección: https://www.makerguides.com/es/paj7620u2-gesture-sensor-with-arduino-es/

[3] A. S. Crandall. «RevEng PAJ7620 Documentation.» Biblioteca Arduino/ESP32 para el sensor de gestos PAJ7620U2. Consultado el 20 de mayo de 2026. dirección: https://acrandal.github.io/RevEng_PAJ7620/latest/index.html

[4] A. S. Crandall, *RevEng_PAJ7620: PAJ7620 gesture sensor Arduino driver and user API*, https://github.com/acrandal/RevEng_PAJ7620, Consultado el 20 de mayo de 2026, 2021.

[5] L. Llamas. «Máquina de estados finitos en Arduino.» Tutorial sobre implementación de máquinas de estados finitos (FSM) en Arduino y plataformas compatibles como ESP32. Consultado el 20 de mayo de 2026. dirección: https://www.luisllamas.es/maquina-de-estados-finitos-arduino/